In [1]:
```
import pandas as pd
df = pd.read_csv("../data/Telco-Customer-Churn.csv")
df.head()
```

Out[1]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multipl
0	7590-VHVEG	Female	0	Yes	No	1	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	No
4	9237-HQITU	Female	0	No	No	2	Yes	

5 rows × 21 columns

In [2]:
```
df.shape
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   str
 1   gender            7043 non-null   str
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   str
 4   Dependents        7043 non-null   str
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   str
 7   MultipleLines     7043 non-null   str
 8   InternetService   7043 non-null   str
 9   OnlineSecurity    7043 non-null   str
 10  OnlineBackup      7043 non-null   str
 11  DeviceProtection  7043 non-null   str
 12  TechSupport       7043 non-null   str
 13  StreamingTV       7043 non-null   str
 14  StreamingMovies   7043 non-null   str
 15  Contract          7043 non-null   str
 16  PaperlessBilling  7043 non-null   str
 17  PaymentMethod     7043 non-null   str
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   str
 20  Churn             7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

There is a issue on the column TotalCharges as it is a numerical filed but the data type is str which is wrong. So it need to further analysis

In [3]:
```
df.describe()
```

Out[3]:

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

In [4]:
```
blank = df["TotalCharges"].str.strip() == ""
print(blank.sum())
```

```
11
```

TotalCharges have 11 empty or blank space that need to fix with numeric value

In [5]:
```
df["Churn"].value_counts(normalize=True)
```

Out[5]:
```
Churn
No     0.73463
Yes    0.26537
Name: proportion, dtype: float64
```

So about 26% customer churn and 73% stayed

In [6]:
```
nums = pd.to_numeric(df["TotalCharges"], errors="coerce")
print(nums.isna().sum())
```

```
11
```

So TotalCharges is filled with the approprited value so our string value become actaul float and empty value will Nan

In [7]:
```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce").fillna(0)
print(df["TotalCharges"].dtype)
print(df["TotalCharges"].isna().sum())
```

```
float64
0
```

So filled it with 0. Now the we need to remove the unnessary coloum such as customer id as it won't have any significat to predict the churn

In [8]:
```
df = df.drop(columns="customerID")
```

In [9]:
```
df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})
```

Need to convert the Churn "YES" "NO" value to binary for better model understanding

In [10]:
```
df.head()
```

Out[10]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inter
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

In [11]:
```
df.isna().sum()
```

Out[11]:
```
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

The data is almost okay as no missing value in any colum is here so far

In [12]:
```
df.isna().sum().sum()
```

Out[12]:
```
np.int64(0)
```

In [13]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 20 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   gender            7043 non-null   str    
 1   SeniorCitizen     7043 non-null   int64  
 2   Partner           7043 non-null   str    
 3   Dependents        7043 non-null   str    
 4   tenure            7043 non-null   int64  
 5   PhoneService      7043 non-null   str    
 6   MultipleLines     7043 non-null   str    
 7   InternetService   7043 non-null   str    
 8   OnlineSecurity    7043 non-null   str    
 9   OnlineBackup      7043 non-null   str    
 10  DeviceProtection  7043 non-null   str    
 11  TechSupport       7043 non-null   str    
 12  StreamingTV       7043 non-null   str    
 13  StreamingMovies   7043 non-null   str    
 14  Contract          7043 non-null   str    
 15  PaperlessBilling  7043 non-null   str    
 16  PaymentMethod     7043 non-null   str    
 17  MonthlyCharges    7043 non-null   float64
 18  TotalCharges      7043 non-null   float64
 19  Churn             7043 non-null   int64  
dtypes: float64(2), int64(3), str(15)
memory usage: 1.1 MB
```

EDA SECTION

The goal of this project is to predict which telecom customers are likely to churn, and then to recommend which customers to target with retention offers and how much to spend on them. EDA is where we learn what the data says about churn before building any model. In this section we will check the target balance. About 26.5% of customers churned, so the classes are imbalanced. This is why we will judge models with ROC-AUC and recall instead of plain accuracy. Compare churners with non-churners on tenure, contract type, internet service, payment method and monthly charges, to see which customer groups leave more often. Look for patterns that guide modelling and strategy, such as whether new customers or month-to-month contracts churn more. These patterns give us hypotheses to test with the models and later support the recommendation.

In [14]:
```
for col in df.columns:
    print(f"{col:20s} unique values: {df[col].nunique()}")
```

```
gender              unique values: 2
SeniorCitizen       unique values: 2
Partner             unique values: 2
Dependents          unique values: 2
tenure              unique values: 73
PhoneService        unique values: 2
MultipleLines       unique values: 3
InternetService     unique values: 3
OnlineSecurity      unique values: 3
OnlineBackup        unique values: 3
DeviceProtection    unique values: 3
TechSupport         unique values: 3
StreamingTV         unique values: 3
StreamingMovies     unique values: 3
Contract            unique values: 3
PaperlessBilling    unique values: 2
PaymentMethod       unique values: 4
MonthlyCharges      unique values: 1585
TotalCharges        unique values: 6531
Churn               unique values: 2
```

The dataset has 20 columns after customerID was removed.

- 16 columns have only 2 to 4 distinct values, so they are categorical: gender, SeniorCitizen, Partner, Dependents, PhoneService, MultipleLines, InternetService, OnlineSecurity
- OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling and PaymentMethod.
- Three columns have many distinct values, so they are numeric: tenure (73), MonthlyCharges (1,585) and TotalCharges (6,531).
- The target is Churn, a binary column.
- Eight columns have 3 values instead of 2 (MultipleLines, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, plus InternetService and Contract). The labels of the extra value still need to be inspected.
- SeniorCitizen is stored as a number but has only 2 values, so it will be treated as a category.

In [15]:
```
for col in df.columns:
    if df[col].nunique() <= 4:
        print(col, "->", df[col].unique().tolist())
```

```
gender -> ['Female', 'Male']
SeniorCitizen -> [0, 1]
Partner -> ['Yes', 'No']
Dependents -> ['No', 'Yes']
PhoneService -> ['No', 'Yes']
MultipleLines -> ['No phone service', 'No', 'Yes']
InternetService -> ['DSL', 'Fiber optic', 'No']
OnlineSecurity -> ['No', 'Yes', 'No internet service']
OnlineBackup -> ['Yes', 'No', 'No internet service']
DeviceProtection -> ['No', 'Yes', 'No internet service']
TechSupport -> ['No', 'Yes', 'No internet service']
StreamingTV -> ['No', 'Yes', 'No internet service']
StreamingMovies -> ['No', 'Yes', 'No internet service']
Contract -> ['Month-to-month', 'One year', 'Two year']
PaperlessBilling -> ['Yes', 'No']
PaymentMethod -> ['Electronic check', 'Mailed check', 'Bank transfer (automatic)',
'Credit card (automatic)']
Churn -> [0, 1]
```

Contract type is likely to be one of the strongest predictors of churn. For the business recommendation, month-to-month customers are the natural group to target, because that is where most churn happens. One possible retention offer is a discount for moving to a longer contract.

In [16]:
```
print(df.duplicated().sum())
```

```
22
```

In [17]:
```
dups = df[df.duplicated(keep=False)].sort_values(list(df.columns))
print(dups.shape)
dups.head(10)
```

```
(42, 20)
```

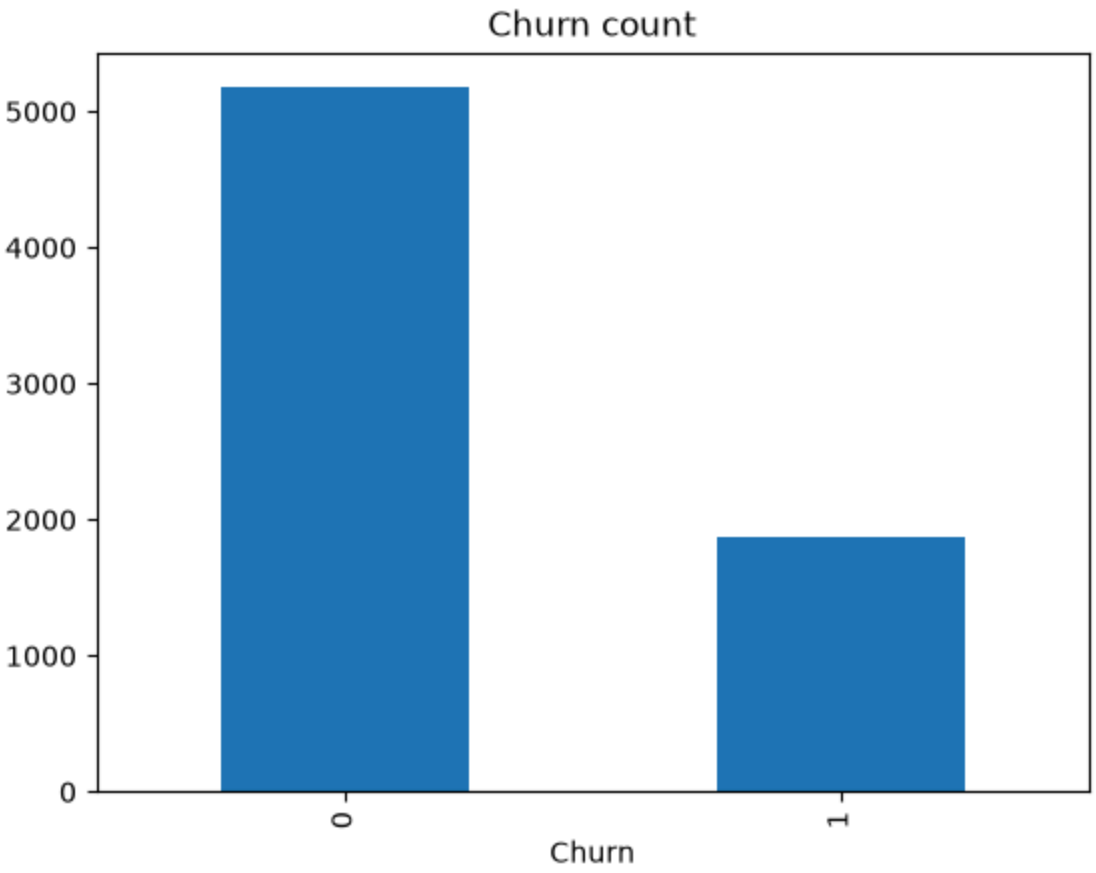
Out[17]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
6491	Female	0	No	No	1	Yes	No	
6764	Female	0	No	No	1	Yes	No	
4495	Female	0	No	No	1	Yes	No	
6267	Female	0	No	No	1	Yes	No	
5522	Female	0	No	No	1	Yes	No	
5759	Female	0	No	No	1	Yes	No	
542	Female	0	No	No	1	Yes	No	
1491	Female	0	No	No	1	Yes	No	
5170	Female	0	No	No	1	Yes	No	
6774	Female	0	No	No	1	Yes	No	

In [18]:
```
df["Churn"].value_counts()
df["Churn"].value_counts(normalize=True).round(3)

df["Churn"].value_counts().plot.bar(title="Churn count")
```

Out[18]:
```
<Axes: title={'center': 'Churn count'}, xlabel='Churn'>
```



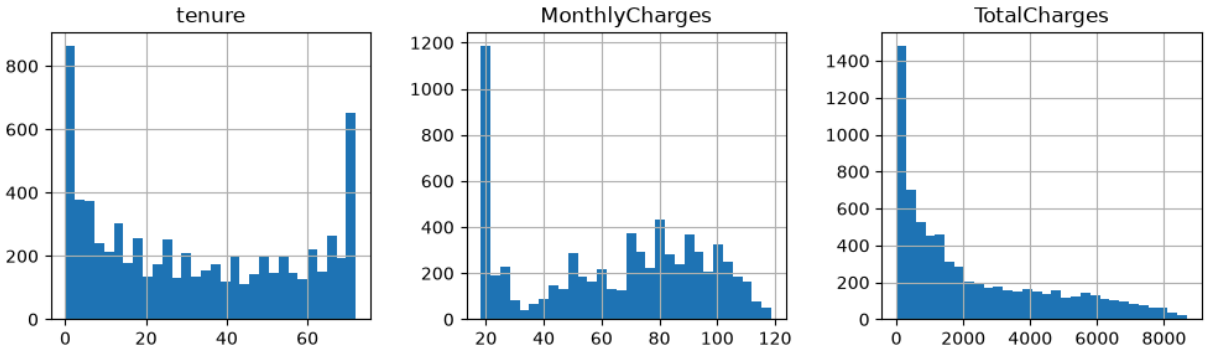
In [19]:
```
df[["tenure", "MonthlyCharges", "TotalCharges"]].describe().round(2)
```

Out[19]:

	tenure	MonthlyCharges	TotalCharges
count	7043.00	7043.00	7043.00
mean	32.37	64.76	2279.73
std	24.56	30.09	2266.79
min	0.00	18.25	0.00
25%	9.00	35.50	398.55
50%	29.00	70.35	1394.55
75%	55.00	89.85	3786.60
max	72.00	118.75	8684.80

In [20]:
```
df[["tenure", "MonthlyCharges", "TotalCharges"]].hist(bins=30, figsize=(12, 3), lay
```

Out[20]:
```
array([[<Axes: title={'center': 'tenure'}>,
        <Axes: title={'center': 'MonthlyCharges'}>,
        <Axes: title={'center': 'TotalCharges'}>]], dtype=object)
```

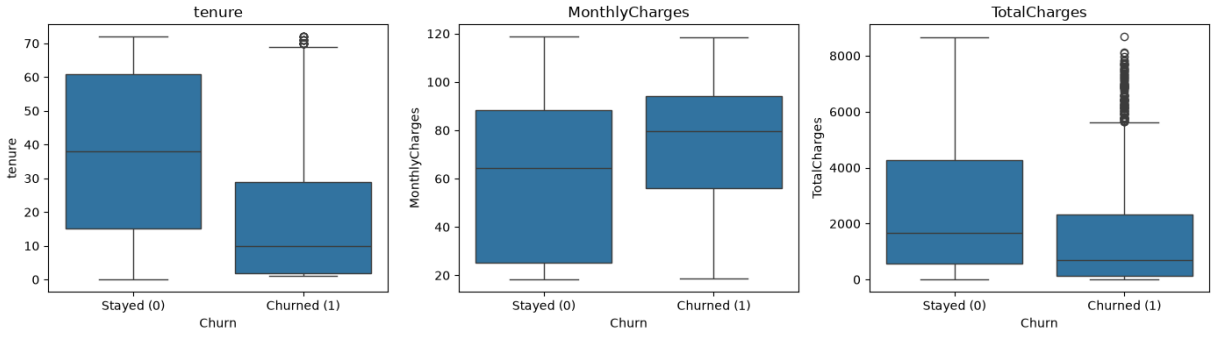


In [21]:
```
import seaborn as sns
import matplotlib.pyplot as plt

num_cols = ["tenure", "MonthlyCharges", "TotalCharges"]

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, col in zip(axes, num_cols):
    sns.boxplot(data=df, x="Churn", y=col, ax=ax)
    ax.set_xticklabels(["Stayed (0)", "Churned (1)"])
    ax.set_title(col)
plt.tight_layout()
plt.show()
```

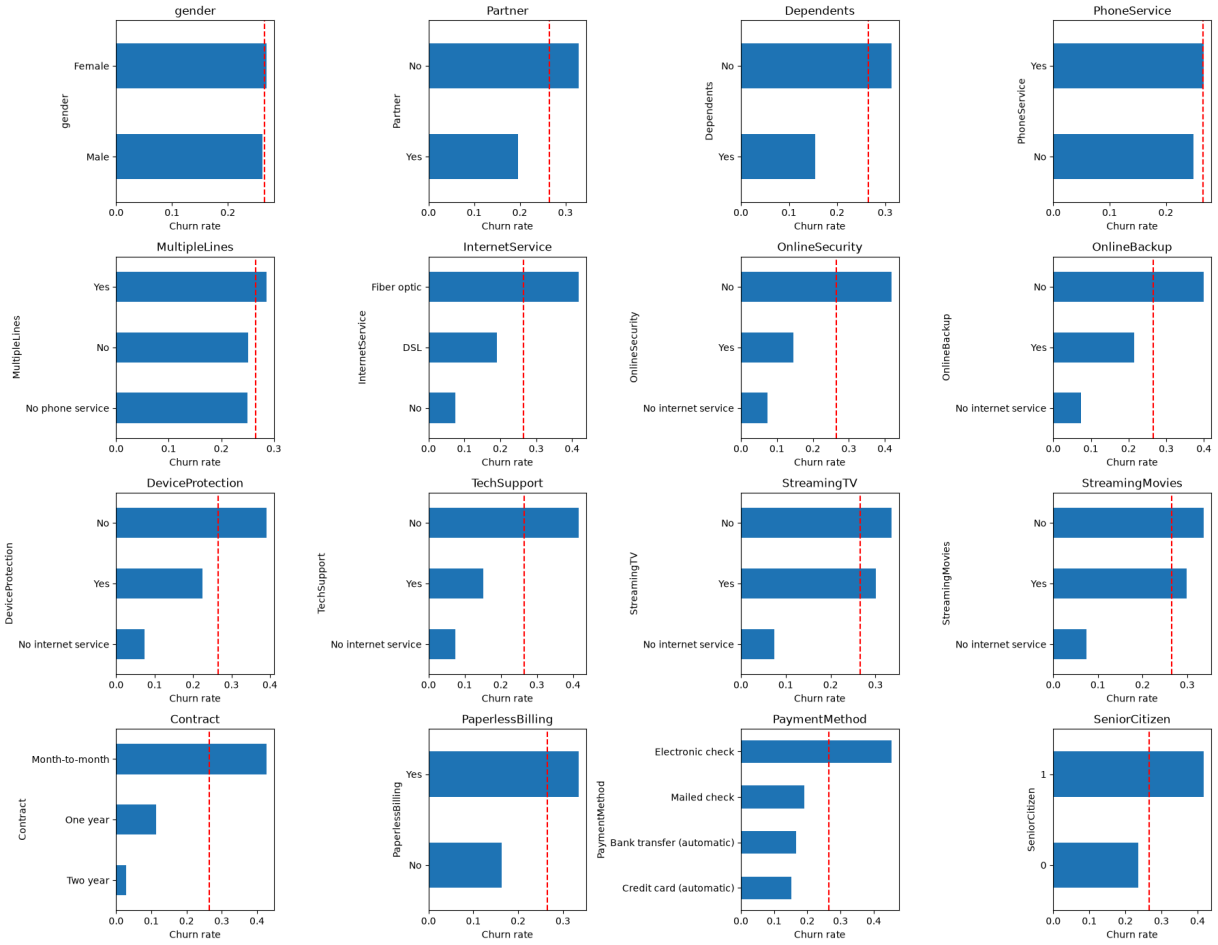
```
C:\Users\amahf\AppData\Local\Temp\ipykernel_3156\2281203210.py:9: UserWarning: set_t
icklabels() should only be used with a fixed number of ticks, i.e. after set_ticks()
or using a FixedLocator. Otherwise, ticks may be mislabeled.
  ax.set_xticklabels(["Stayed (0)", "Churned (1)"])
C:\Users\amahf\AppData\Local\Temp\ipykernel_3156\2281203210.py:9: UserWarning: set_t
icklabels() should only be used with a fixed number of ticks, i.e. after set_ticks()
or using a FixedLocator. Otherwise, ticks may be mislabeled.
  ax.set_xticklabels(["Stayed (0)", "Churned (1)"])
C:\Users\amahf\AppData\Local\Temp\ipykernel_3156\2281203210.py:9: UserWarning: set_t
icklabels() should only be used with a fixed number of ticks, i.e. after set_ticks()
or using a FixedLocator. Otherwise, ticks may be mislabeled.
  ax.set_xticklabels(["Stayed (0)", "Churned (1)"])
```



In [22]:
```
cat_cols = df.select_dtypes(include="str").columns.tolist() + ["SeniorCitizen"]

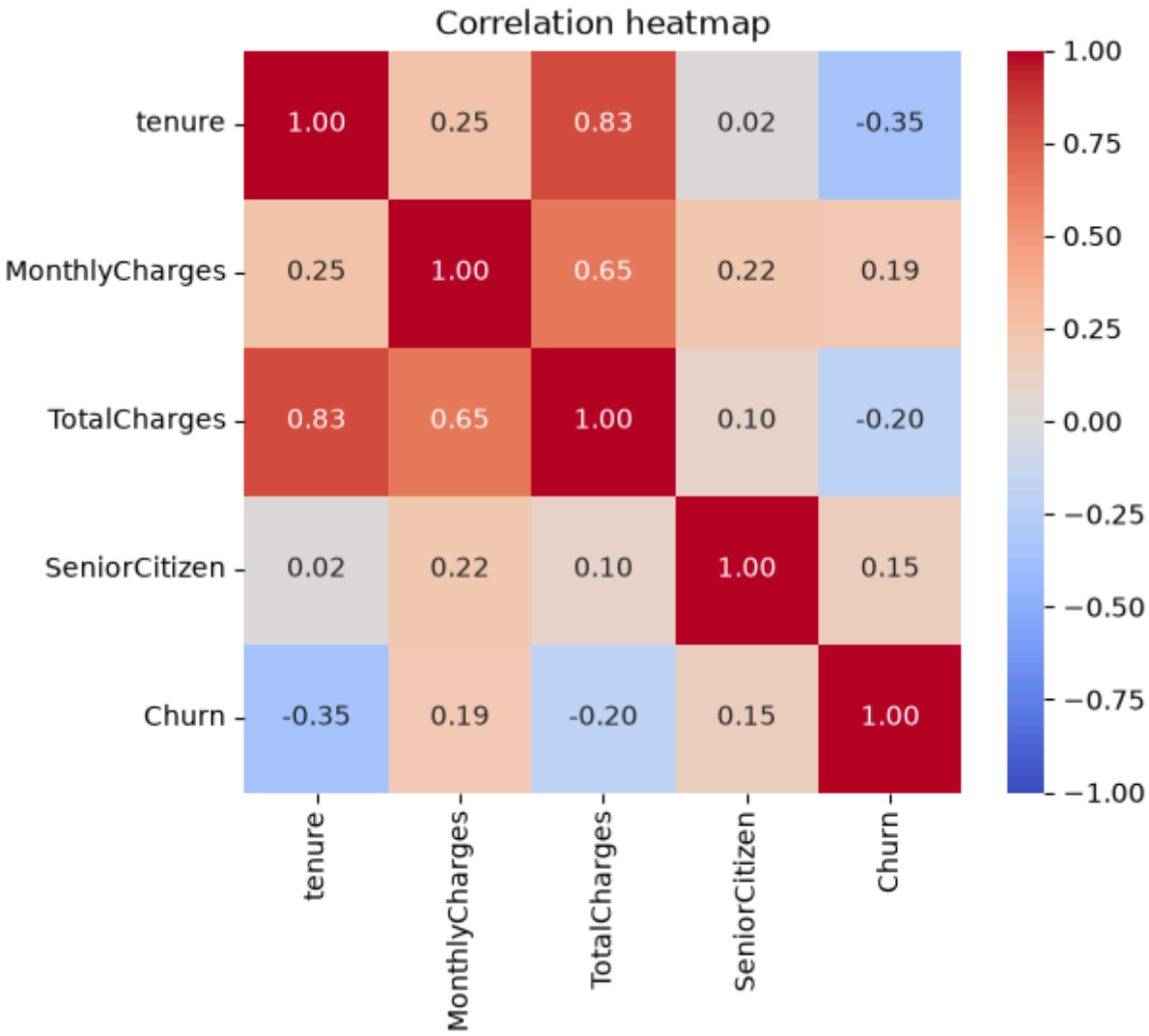
fig, axes = plt.subplots(4, 4, figsize=(18, 14))
for ax, col in zip(axes.ravel(), cat_cols):
    df.groupby(col)["Churn"].mean().sort_values().plot.barh(ax=ax)
    ax.axvline(df["Churn"].mean(), color="red", linestyle="--")
    ax.set_title(col)
    ax.set_xlabel("Churn rate")
```

```
plt.tight_layout()
plt.show()
```



In [23]:
```
corr = df[["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen", "Churn"]].c

plt.figure(figsize=(6, 5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation heatmap")
plt.show()
```



There is some feature overlap like tenure and TotalCharges which is obious

Data preparation for model training

In [24]:
```
df_model = df.copy()
```

In [25]:
```
df_model = df_model.replace({"No internet service": "No",
                             "No phone service": "No"})
```

In [26]:
```
for col in ["MultipleLines", "OnlineSecurity", "TechSupport"]:
    print(df_model[col].value_counts(), "\n")
```

```
MultipleLines
No     4072
Yes    2971
Name: count, dtype: int64

OnlineSecurity
No     5024
Yes    2019
Name: count, dtype: int64

TechSupport
No     4999
Yes    2044
Name: count, dtype: int64
```

In [27]:
```
X = df_model.drop(columns="Churn")
y = df_model["Churn"]

print(X.shape, y.shape)
```

```
(7043, 19) (7043,)
```

In [28]:
```
X_encoded = pd.get_dummies(X, drop_first=True, dtype=int)
print(X_encoded.shape)
X_encoded.head()
```

```
(7043, 23)
```

Out[28]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	gender_Male	Partner_Yes	Depenc
0	0	1	29.85	29.85	0	1	
1	0	34	56.95	1889.50	1	0	
2	0	2	53.85	108.15	1	0	
3	0	45	42.30	1840.75	1	0	
4	0	2	70.70	151.65	0	0	

5 rows × 23 columns

One hot encoding finish as model only understand numerical data so it is needed to converted the text field to num

In [29]:
```
X_encoded.dtypes.value_counts()
```

Out[29]:
```
int64      21
float64     2
Name: count, dtype: int64
```

In [30]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X_encoded, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

In [31]:
```
print("Train:", X_train.shape, "Test:", X_test.shape)
print("Churn rate in train:", round(y_train.mean(), 3))
print("Churn rate in test: ", round(y_test.mean(), 3))
```

```
Train: (5634, 23) Test: (1409, 23)
Churn rate in train: 0.265
Churn rate in test:  0.265
```

In [32]:
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

log_reg = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(class_weight="balanced", max_iter=1000))
])
```

In [33]:
```
scores = cross_val_score(log_reg, X_train, y_train, cv=5, scoring="roc_auc")

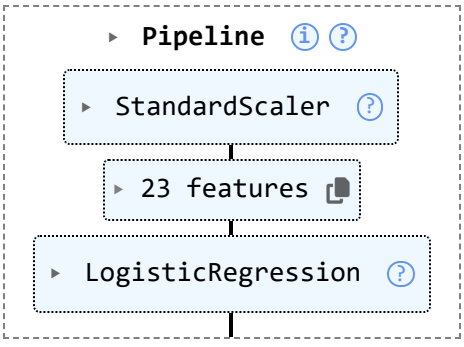
print("ROC-AUC per fold:", scores.round(3))
print("Average ROC-AUC: ", scores.mean().round(3))
```

```
ROC-AUC per fold: [0.864 0.852 0.852 0.835 0.824]
Average ROC-AUC:  0.845
```

An AUC of 0.5 is random guessing and 1.0 is perfect ranking, so 0.845 means the model is good at ranking customers by churn risk. If pick one churner and one non-churner at random, the model gives the churner the higher risk score about 84.5% of the time. The fold scores range from 0.824 to 0.864, so performance is fairly stable and doesn't depend heavily on which rows land in the training set. Logistic regression achieves a cross-validated ROC-AUC of 0.845 (±~0.015 across folds), meaning it ranks churners above non-churners about 84.5% of the time, which is good discrimination for a baseline.

In [34]:
```
log_reg.fit(X_train, y_train)
```

Out[34]:



Cross-validation only scored the model and did not keep a fitted version, so the baseline pipeline is now refit on the entire training set (80% of the data). The pipeline scales the 23 encoded features with StandardScaler, then fits a logistic regression with class_weight="balanced" so that churners, the minority class, are not ignored. This fitted model will be evaluated once on the held-out test set.

In [35]:
```
from sklearn.metrics import roc_auc_score, classification_report, confusion_matrix,
y_pred = log_reg.predict(X_test)
y_prob = log_reg.predict_proba(X_test)[:, 1]
print("Test ROC-AUC:", round(roc_auc_score(y_test, y_prob), 3))
print(classification_report(y_test, y_pred, target_names=["Stayed", "Churned"]))
```

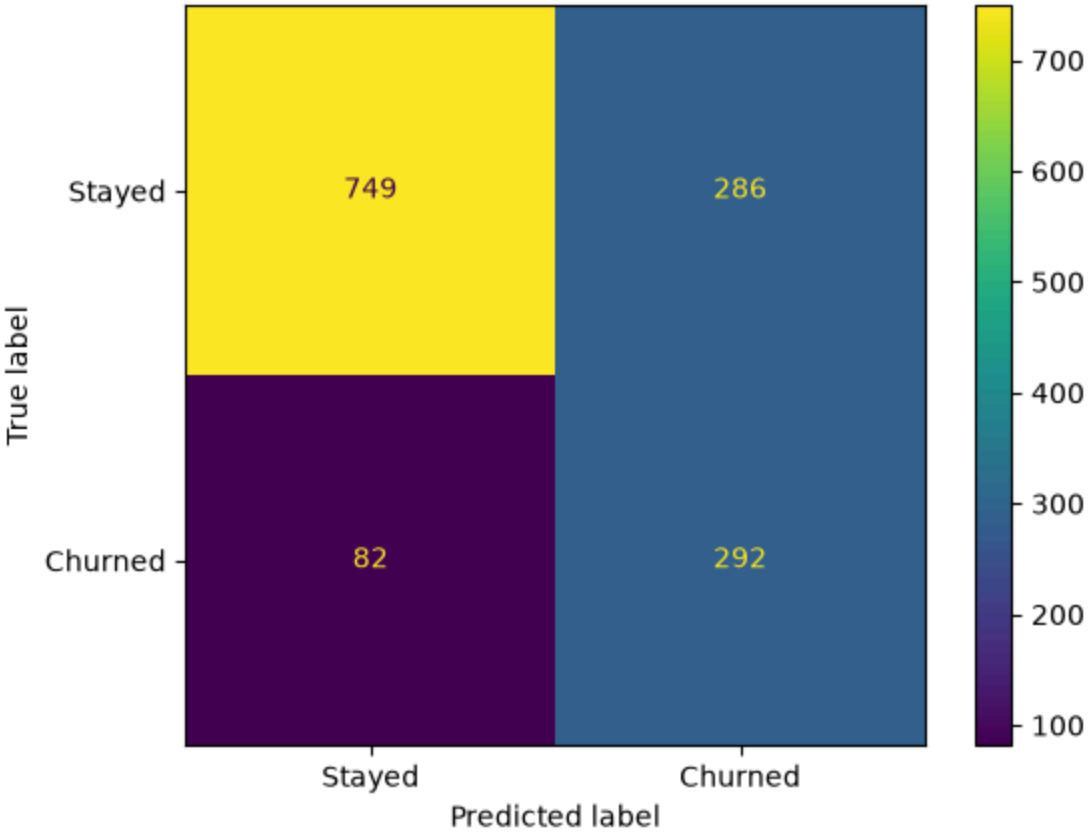
```
Test ROC-AUC: 0.841
              precision    recall  f1-score   support

      Stayed       0.90      0.72      0.80      1035
     Churned       0.51      0.78      0.61       374

    accuracy                           0.74      1409
   macro avg       0.70      0.75      0.71      1409
weighted avg       0.80      0.74      0.75      1409
```

On the held-out test set, the logistic regression baseline reaches a ROC-AUC of 0.841, in line with the cross-validated 0.845, so there is no sign of overfitting. The model identifies 78% of churners (recall) but only 51% of the customers it flags actually churn (precision), because class_weight="balanced" deliberately favours catching churners over avoiding false alarms. Accuracy (0.74) is not a meaningful measure here since a model that predicted “stays” for everyone would score about 73%.

In [36]:
```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred,display_labels=["Stayed", "C
plt.show()
```



The confusion matrix shows that the model correctly identifies 292 of the 374 churners (recall 78%) and misses 82. To achieve this, it flags 286 loyal customers as at-risk, so only about half of flagged customers (51% precision) would actually have left.

In [37]:
```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

rf = RandomForestClassifier(
    n_estimators=300,
    min_samples_leaf=5,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)
```

Random forest and a gradient boosting model are compared against the logistic regression baseline. The random forest uses 300 trees with a minimum of 5 customers per leaf to limit overfitting, and class_weight="balanced"

In [38]:
```
gb = GradientBoostingClassifier(random_state=42)
```

In [39]:
```
models = {
    "Logistic Regression": log_reg,
    "Random Forest": rf,
    "Gradient Boosting": gb
}

for name, m in models.items():
```

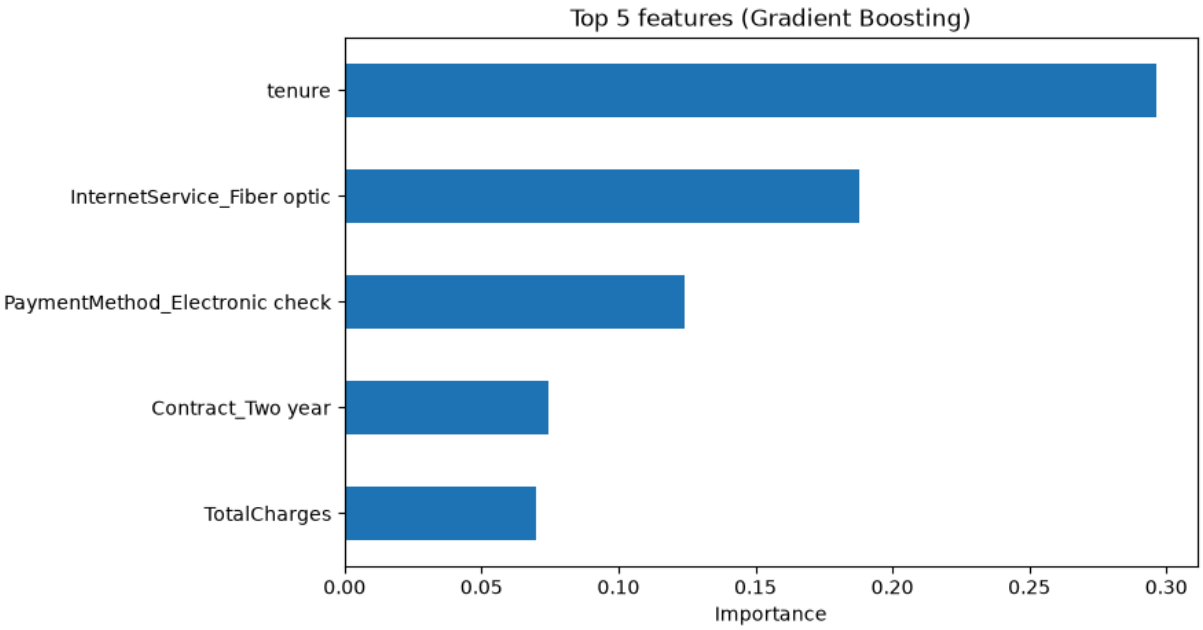
```
    s = cross_val_score(m, X_train, y_train, cv=5, scoring="roc_auc")
    print(f"{name:20s} mean AUC = {s.mean():.3f}   folds: {s.round(3)}")
```

```
Logistic Regression  mean AUC = 0.845   folds: [0.864 0.852 0.852 0.835 0.824]
Random Forest        mean AUC = 0.844   folds: [0.86  0.854 0.843 0.837 0.827]
Gradient Boosting    mean AUC = 0.847   folds: [0.862 0.857 0.849 0.839 0.827]
```

All three models are compared using the same 5-fold stratified cross-validation on the training set, scored by ROC-AUC. Using identical folds ensures the differences between models reflect the models, not the data splits. Only the training data is used here, so the test set remains untouched for final evaluation. Random forest (0.844) and gradient boosting (0.847) perform no better than logistic regression (0.845). The 0.003 spread is much smaller than the variation between folds (~0.015), so there is no meaningful difference. The simpler, more interpretable logistic regression is therefore preferred.

In [40]:
```
gb.fit(X_train, y_train)

importances = pd.Series(gb.feature_importances_, index=X_train.columns)
importances.sort_values().tail(5).plot.barh(figsize=(8, 5), title="Top 5 features (
plt.xlabel("Importance")
plt.show()
```

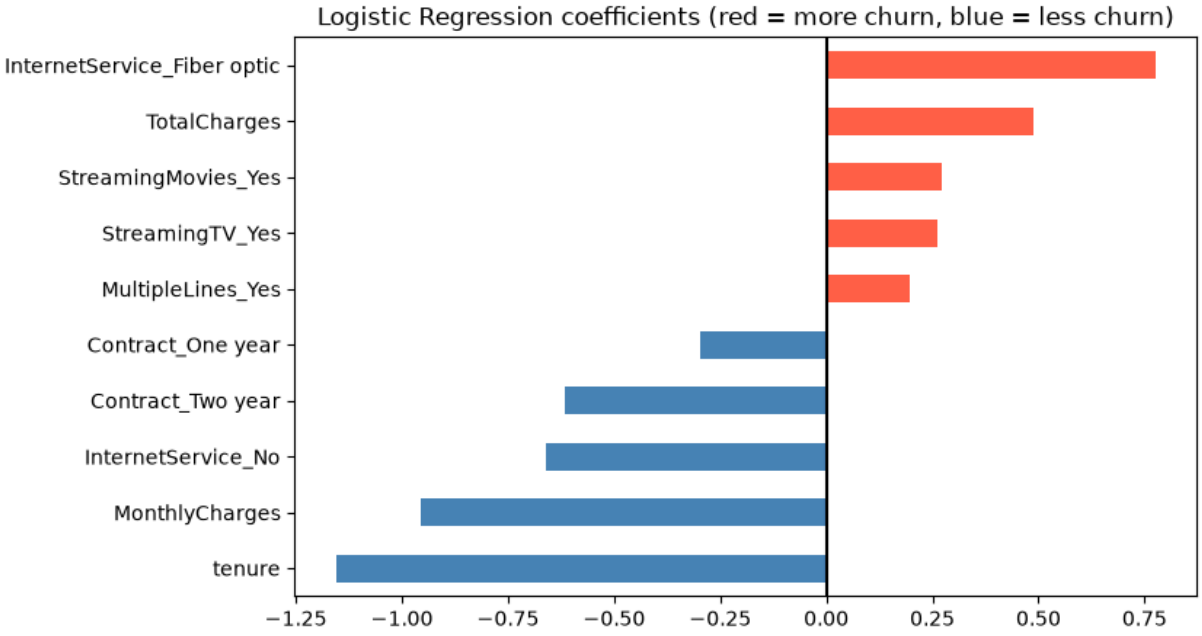


Gradient boosting ranks tenure as by far the most important feature (≈0.30), followed by fiber optic internet (≈0.19), electronic check payment (≈0.12), two-year contract (≈0.07) and TotalCharges (≈0.07). TotalCharges largely overlaps with tenure, since both grow as a customer stays longer. Importance shows which features the model relies on, not the direction of the effect. The direction (short tenure, fiber and electronic check raise churn; long contracts lower it) comes from the EDA churn-rate charts and the logistic regression coefficients.

The customers most at risk are new customers on a month-to-month contract, using fiber internet and paying by electronic check. To reduce churn, the company need to focus on

new customers with a discount to move to a one or two-year contract, since long contracts have the lowest churn. Encourage automatic payment (bank transfer or card) with a small bill credit. Look into fiber customer s satisfaction, because they leave far more often than DSL customers. Needs to focus more on retention effort on the first year, when customers are most likely to leave.

In [41]:
```
coefs = pd.Series(log_reg.named_steps["model"].coef_[0], index=X_train.columns)
coefs = coefs.sort_values()
top = pd.concat([coefs.head(5), coefs.tail(5)])
top.plot.barh(figsize=(8, 5), color=(top > 0).map({True: "tomato", False: "steelblu
plt.axvline(0, color="black")
plt.title("Logistic Regression coefficients (red = more churn, blue = less churn)")
plt.show()
```



Bar to the right (red, positive): this feature raises churn risk. Bar to the left (blue, negative): this feature lowers churn risk. Longer bar equal to stronger effect. Data is scale in the pipeline, so the bars are comparable to each other. The logistic regression coefficients (on standardised features) show the direction of each effect. Short tenure and fiber optic internet are the strongest drivers of churn, and one- and two-year contracts are the strongest protective factors relative to month-to-month, consistent with the gradient boosting importances. Streaming services and multiple lines also add to churn risk. The coefficients for MonthlyCharges and TotalCharges should not be interpreted individually: they are highly correlated with tenure and the service features, which explains why their signs appear counter-intuitive.

The business question is not only who is likely to churn, but for whom a retention offer pays off. Using gradient boosting's churn probabilities, the expected net benefit of an offer is calculated per customer as P(churn) × offer success rate × monthly charges × months retained − offer cost, under illustrative assumptions of a $50 offer, a 30% success rate and 12 months of retained revenue. Three targeting strategies are compared on the test set:

everyone, month-to-month customers only, and customers with positive expected net benefit

In [42]:
```
offer_cost, success_rate, months_saved = 50, 0.30, 12

test = df.loc[X_test.index].copy()
test["p_churn"] = gb.predict_proba(X_test)[:, 1]
test["net_benefit"] = test["p_churn"] * success_rate * test["MonthlyCharges"] * mon

results = pd.DataFrame({
    "Strategy": ["Target everyone", "Month-to-month only", "Model: positive expecte
    "Customers targeted": [len(test),
                           (test["Contract"] == "Month-to-month").sum(),
                           (test["net_benefit"] > 0).sum()],
    "Expected net benefit ($)": [test["net_benefit"].sum(),
                                 test.loc[test["Contract"] == "Month-to-month", "ne
                                 test.loc[test["net_benefit"] > 0, "net_benefit"].s
})
results["Spend ($)"] = results["Customers targeted"] * offer_cost
results.round(0)
```

Out[42]:

	Strategy	Customers targeted	Expected net benefit ($)	Spend ($)
0	Target everyone	1409	29411.0	70450
1	Month-to-month only	773	47188.0	38650
2	Model: positive expected value	625	56777.0	31250

The model-based rule is the best strategy. It targets 625 customers (44% of the test set) for USD 31,250 and yields an expected net benefit of about USD 56,800, roughly double the USD 29,400 from targeting everyone, which costs more than twice as much (USD 70,450). It also beats the simple month-to-month rule (USD 47,200 net at USD 38,650), because it excludes low-value customers who are not worth the offer. The assumptions are illustrative; the company should substitute its real offer cost and success rate, but the decision rule stays the same.

Conclusion

Customers who are new, on month-to-month contracts, using fiber internet and paying by electronic check are the most likely to leave, and a simple model identifies them reasonably well (AUC ≈ 0.84). Rather than sending offers to everyone, the company should target only customers whose churn risk and monthly revenue make an offer profitable. Under illustrative assumptions this means about 44% of customers, a spend of roughly $50 per targeted customer, and about twice the net benefit of a blanket campaign. Supporting actions are discounts for longer contracts, a bill credit for automatic payment, and a review of fiber customer satisfaction.

In [ ]:
```
```